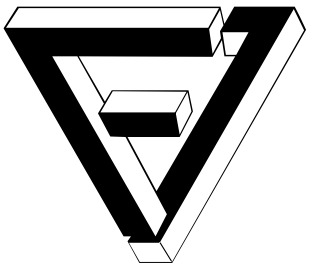
Masaryk University
Faculty of Informatics



Side-Effect Analysis Using a Static Single Use Program Representation

Master's Thesis

Samuel Malec

Brno, 2026

Declaration

I hereby declare that this thesis is my original work and that I developed it independently. I have properly cited all sources and literature used in this thesis and provided complete references to them.

Supervisor: TBD

Acknowledgements

Write your acknowledgements here.

Abstract

Write your abstract here.

Keywords

keyword, keyword

Contents

1 Introduction 1

2 Background 2
- 2.1 Side Effect Analysis . . . 2
- 2.2 Static Single Use Form . . . 2
- 2.3 Sea of Noddes . . . 2
- 2.4 GraalVM and Graal IR . . . 2

3 Explicit Effects 3
- 3.1 A language with tokens . . . 4
- 3.2 Semantics . . . 8
- 3.3 Partitions . . . 10
- 3.4 Signatures . . . 13
- 3.5 What the tokens guarantee . . . 16
- 3.6 What the tokens license . . . 18
- 3.7 Fresh objects . . . 24
- 3.8 Concurrency . . . 25
- 3.9 What a kill set guarantees . . . 26
- 3.10 Summary . . . 27

4 Analysis and Optimization of the Explict Effect Form 29

5 From Graal IR and back 30

6 Evaluation 31

7 Conclusion 32

Bibliography 33

I Introduction

2 Background

In this chapter we will introduce the necessary terms that underpin the core ideas presented in this thesis.

2.1 Side Effect Analysis

2.2 Static Single Use Form

2.3 Sea of Noddes

2.4 GraalVM and Graal IR

3 Explicit Effects

An optimizing compiler may reorder, merge and remove operations on memory only when nothing in between can interfere with them. Within a method it can usually tell. A call it has not inlined is another matter: Graal [6], like most compilers, takes a call to read and write all of memory, so it forwards no load across a call and removes no store before one. The running example of this chapter is a method of the program `Aliasing`, one of those the thesis evaluates (Chapter 6):

```
static int forwardAcrossCallOnOtherObject(Counter counter, Counter other, Link list) {
    counter.value = 5;
    countInto(other, list);   // writes Counter.value, but only of other
    return counter.value;
}
```

The call writes the field `value` of the counter it is given, and the program only ever gives it a counter other than the one it passes this method as `counter`. The load after the call therefore reads 5. Graal keeps the load, because in its model the call may write anything.

This chapter makes the effects of each operation explicit, as values. Every method takes extra parameters and gives back extra results, *tokens*: one for each part of memory, a *partition*, that it or anything it calls may touch. Every operation on memory takes the token of its partition and gives back a new one, which the next operation on that partition takes. The order of the operations on memory becomes data flow. In a program in which every value is used exactly once, the tokens of a partition form a chain, and two operations are ordered only when a chain connects them. In the example, with a partition for each field of each set of objects the points-to analysis tells apart, the call takes the token of the counters the program passes as `other`, but not the token of the one it passes as `counter`. The load's token comes straight from the store, so the load can be replaced by the 5.

Section 3.1 defines a small language with tokens, the core of graal.ct, the dialect of Cthulhu in which the implementation represents Graal IR, and Section 3.2 its semantics. Section 3.3 defines partitions and the four partitionings the implementation offers, and Section 3.4 the signatures that say which tokens a method takes. Section 3.5 proves what the tokens guarantee: an operation leaves a partition alone unless it takes its token, and two operations that share no token can be swapped. Section 3.6 proves which rewrites the tokens license: forwarding a load and removing a store, with exceptions as Java has them, and that a finer partitioning licenses every rewrite a

τ	::=	$\mathsf{int} \mid \mathsf{bool} \mid \mathsf{ref}$	values
	$\mid$	$\mathsf{heap}_P \qquad (P \in \Pi)$	the token of partition P
ι	::=	$\bar{x} \coloneqq op(\bar{y})$	an operation on values
	$\mid$	$y_1, y_2 \coloneqq \mathsf{dup}\, x \mid \mathsf{drop}\, x$	copy, discard a value
	$\mid$	$h' \coloneqq \mathsf{move}\, h$	pass a token on
	$\mid$	$x, h' \coloneqq \mathsf{get}^P_f(h, y)$	load field f of y
	$\mid$	$h' \coloneqq \mathsf{set}^P_f(h, y, z)$	store z into field f of y
	$\mid$	$h' \coloneqq \mathsf{init}^P_f(h, y, z) \mid \mathsf{init}_f(y, z)$	give field f of a new y its first value
	$\mid$	$x, h' \coloneqq \mathsf{new}_C(h)$	allocate an object of class C
	$\mid$	$x, h' \coloneqq \mathsf{exception}(h)$	make the exception of a check that failed
	$\mid$	$\bar{x}, e, \bar{h}' \coloneqq m(\bar{y}, \bar{h})$	call m; e is what it threw, or null
	$\mid$	$\bar{h}' \coloneqq \mathsf{guard}_b(\bar{h}, y)$	go on in b, which leaves, when y holds
κ	::=	$\mathsf{jump}\, b(\bar{y})$	go on in block b
	$\mid$	$\mathsf{if}\, y\, \mathsf{then}\, b_1\, \mathsf{else}\, b_2\, (\bar{y})$	branch
	$\mid$	$\mathsf{return}(\bar{y}) \mid \mathsf{throw}(e, \bar{h})$	leave the method, normally or with e
	$\mid$	$\mathsf{trap}(\bar{h})$	stop the program
B	::=	$b(\bar{x} : \bar{\tau}) = \iota_1 \ldots \iota_n;\ \kappa$	a block

Figure 3.1: The core of graal.ct. Variables x, y, z hold values and h tokens; $\bar{x}$ is a sequence of them. A method is a set of blocks, one of which is its entry.

coarser one does. The next three sections give the conditions under which an effect may be left out: initializing a new object (Section 3.7) and synchronization (Section 3.8), and what the same facts guarantee when they go back to Graal as kill sets (Section 3.9). Section 3.10 lists every result with what it assumes and what relies on it.

Each result was also held against the implementation. Where the two disagree, the section says so, and Section 3.10 collects what that turned up.

3.1 A language with tokens

The implementation translates each method of Graal IR into graal.ct (Chapter 5). We reason about a small core of graal.ct, which keeps everything that matters for memory. Figure 3.1 gives its syntax.

A program consists of methods. A method m consists of blocks, one of which is its entry block b_m. It takes values of types $\bar{\tau}_m$ and gives back values of types $\bar{\tau}'_m$, and it takes and gives back one token of each partition in a set T_m, defined in Section 3.4. A block $b(\bar{x} : \bar{\tau}) = \iota_1 \ldots \iota_n;\ \kappa$ binds its parameters $\bar{x}$, runs its instructions in order, and ends with a terminator κ: a jump to another block of the method, a branch to one of two, or leaving the method: normally with return, with

an exception with throw, or by stopping the program with trap, which is what a deoptimization does in ahead-of-time code. Blocks interact only through their parameters, and methods only through calls and memory.

This is the shape of Graal IR once it is scheduled: a basic block becomes a block, a φ-function becomes a parameter of its block, and each edge passes the φ's input for that edge as an argument. That a program in static single assignment form can be read this way, as a set of mutually recursive functions that call each other in tail position, is well known [8, 2].

Each memory instruction is annotated with a partition P. A load $x, h' := \mathsf{get}_f^P(h, y)$ takes the token h of P, reads field f of the object y into x, and gives back the next token of P as h'. A store gives back only the token. An initialization gives a field of an object that was just allocated its first value; it takes a token only if its method takes the token of that field's partition for another reason (Section 3.7). An allocation takes the token of the rest of memory, rest, which holds the allocator's state, and so does exception, which makes the exception a failed check throws.

A call takes and gives back the tokens of the callee's partitions. It also gives back the exception the callee threw, or null, and its block ends with a branch on it: to the block that handles the exception, or leaves the method with it. That is Graal IR's own shape, in which every call has an edge for when it throws. A guard abbreviates such a branch, to a block b every path from which leaves the method: when its condition holds it goes on in b, with the values the branch passed. It takes and gives back a token of every partition of its method. The rules make guards (Section 3.6.4); the translation makes none.

3.1.1 Single use

Definition 3.1 (Static single use)**.** A method is in *static single use* (SSU) form if each of its blocks satisfies the following:

1. every variable is bound once, as a parameter of the block or as a result of one of its instructions;
2. every variable bound in the block is used exactly once after it is bound, by an instruction or by the terminator;
3. dup and drop are only applied to values, never to tokens.

Static single assignment [5] asks only for the first condition, and lets a value be used any number of times. Static single use asks that every use have a value of its own: a value two instructions need is first copied with dup, and a value nobody needs is discarded with drop. What this costs is the copies. What it buys is that the lifetime of a value is explicit, from its definition to its one use.

Instruction	Takes	Gives
$\bar{x} \coloneqq op(\bar{y})$	the operation's values	its values
$y_1, y_2 \coloneqq \mathsf{dup}\, x; \mathsf{drop}\, x$	a value of type $\tau \neq \mathsf{heap}_P$	two of type τ; nothing
$h' \coloneqq \mathsf{move}\, h$	heap_P	heap_P
$x, h' \coloneqq \mathsf{get}_f^P(h, y)$	heap_P, ref	τ_f, heap_P
$h' \coloneqq \mathsf{set}_f^P(h, y, z)$, $h' \coloneqq \mathsf{init}_f^P(h, y, z)$	heap_P, ref, τ_f	heap_P
$\mathsf{init}_f(y, z)$	ref, τ_f	nothing
$x, h' \coloneqq \mathsf{new}_C(h)$, $x, h' \coloneqq \mathsf{exception}(h)$	$\mathsf{heap}_{\mathsf{rest}}$	ref, $\mathsf{heap}_{\mathsf{rest}}$
$\bar{x}, e, \bar{h}' \coloneqq m(\bar{y}, \bar{h})$	$\bar{\tau}_m$, heap_{T_m}	$\bar{\tau}'_m$, ref, heap_{T_m}
$\bar{h}' \coloneqq \mathsf{guard}_b(\bar{h}, y)$	heap_S, bool	heap_S

Figure 3.2: The types of instructions. τ_f is the type of field f, and heap_S for a set S of partitions is one token of each, in a fixed order. Every instruction gives back exactly the tokens it takes.

The third condition is what makes tokens special. A variable that may be neither copied nor discarded is *linear* [7, 19]: it is used exactly once, so at every point of a block exactly one version of each token exists. Threading the state of the world through a program as a linear value is an old idea. Haskell's state threads [9] and Clean's uniqueness types [3] use it to make input, output and mutable arrays purely functional. Here the state is split, one token per partition, and the split is what lets operations on different partitions happen in any order.

3.1.2 Types

Definition 3.2 (Well-typed method)**.** A method m with tokens T_m is *well typed* if every block of m takes, besides its values, one token of each partition in T_m; every instruction is used at its type in Fig. 3.2, with guards over $S = T_m$; the arguments of a jump or a branch have the types of the parameters of the blocks it goes to, and the two blocks of a branch take the same parameters; return gives back values of types $\bar{\tau}'_m$ and a token of each partition in T_m, throw an exception and the tokens, and trap takes the tokens.

Lemma 3.3 (Chains)**.** *Let b be a block of a well-typed method m in SSU form, and $P \in T_m$. The instructions of b that take a token of P, in the order in which they appear, are $\iota_{k_1}, \ldots, \iota_{k_r}$, where ι_{k_1} takes the parameter of b of type heap_P, each $\iota_{k_{i+1}}$ takes the token of P that ι_{k_i} gives, and the terminator takes the one ι_{k_r} gives, or the parameter if $r = 0$.*

Proof. We show that after each instruction exactly one variable of type heap_P is bound and not yet used. Before the first instruction, it is the parameter. An instruction that takes a token of P takes exactly one (Fig. 3.2), and it must be that variable: every other variable of type heap_P bound so far has been used already, and none can be used twice. The instruction gives back exactly one

```
run = λ p0 p1 p2 h ⟶ r exc hout                    ; ref × ref × ref × heap ⟶ int × ref × heap
(
    ref nil? p0 ⟶ v6                                    ; counter == null
    Aliasing_forwardAcrossCallOnOtherObject b1 ⟶ kt      ; b1 throws
    Aliasing_forwardAcrossCallOnOtherObject b2 ⟶ kf
    Aliasing_forwardAcrossCallOnOtherObject frame_0 ⟶ frame
    bool not v6 ⟶ cn
    f_rrrh_irh opt v6 kt ⟶ at
    f_rrrh_irh opt cn kf ⟶ af
    f_rrrh_irh join at af frame ⟶ k
    f_rrrh_irh call k p0 p1 p2 h ⟶ r exc hout
)
frame_0 = λ A B x0 x1 x2 h ⟶ r exc hout
(
    heap fork h ⟶ h_a h_b
    f_rrrh_irh call A x0 x1 x2 h_a ⟶ r_a exc_a hout_a
    f_rrrh_irh call B x0 x1 x2 h_b ⟶ r_b exc_b hout_b
    int join r_a r_b ⟶ r
    ref join exc_a exc_b ⟶ exc
    heap join hout_a hout_b ⟶ hout
)
```

Figure 3.3: The null check at the start of the running example, with one token for all of memory, as the implementation prints it but without the copies of values, which dup makes. `f_rrrh_irh` is the type of a closure that takes three references and a token and gives back an `int`, an exception and a token.

new token of P. An instruction that takes no token of P gives none. After the last instruction, the one remaining token must still be used, and only the terminator is left. □

So in every block the operations on one partition are totally ordered, and the order is the token's data flow. Nothing orders two instructions that take the tokens of different partitions, other than the values one gives the other.

3.1.3 Branches in Cthulhu

Cthulhu has no branch terminator: every block ends with a call. A two-way branch is built from closures, as Fig. 3.3 shows on the running example. opt(c, k) is the closure k when c holds and $\bot$ otherwise, and join(a_1, a_2, g) is a closure that calls the *frame* g with both. The frame forks each token in two, gives one half to each arm, and joins what the two arms give back, where $\bot$ is the unit of join. In every execution exactly one of the arms is not $\bot$, so of the two halves of a token, only one ever reaches an operation: fork is not dup. In terms of linear logic, the two halves form the additive conjunction of the two arms, not the multiplicative one: a block may use either, never both. The core language replaces the encoding with if, which behaves the same.

3.1.4 The rest of graal.ct

graal.ct has more than the core. It has numbers of four sizes and their operations, constants of the program, and static fields, which it accesses through one object per class. It has arrays, whose elements are locations (o, i) and whose accesses aget_K and aset_K are annotated like field accesses: with the partition of the element kind K, or of the elements of kind K of a set of arrays. And it has three kinds of call the core lacks:

- a node of Graal IR that touches memory in a way graal.ct has no operation for, such as an array copy or the initialization of a class, becomes a call of an external method that takes every token of its method;
- a call with several targets becomes a call of an external method that takes the tokens of all of them;
- a call the analysis could not resolve takes every token of the program.

All of them are memory instructions or calls in the sense of this chapter: each takes the tokens of every partition its execution may touch, which is all the results below rely on.

3.2 Semantics

Definition 3.4 (Heaps)**.** Objects are natural numbers, $Obj = \mathbb{N}$, and values are $Val = \mathbb{Z} \cup \{tt, ff\} \cup Obj \cup \{\mathsf{null}, \bullet\}$, where $\bullet$ is the only value a token ever has. A *location* is a field of an object, or the allocator: $Loc = (Obj \times F) \cup \{\mathsf{alloc}\}$, where F is the set of fields, including two header fields, cls and site, which record an object's class and where it was allocated, and which no instruction writes. A *heap* is a finite partial function σ from Loc to Val in which $\sigma(\mathsf{alloc})$ is the number of objects allocated so far. An object o *exists* in σ if $o < \sigma(\mathsf{alloc})$, and then $(o, f) \in \operatorname{dom}\sigma$ for each field f of its class and for the header fields. An *environment* ρ maps variables to values.

Figure 3.4 gives the semantics. The relation is defined inductively, so an invocation that does not terminate has no outcome. An invocation leaves its method normally, with ret, or with an exception, with exc, which carries the heap: a handler up the stack, or the code that prints an uncaught exception, may read it. A call catches its callee's exception in e, and its block branches on e. Two rules are there because Java has them: any allocation may throw `OutOfMemoryError` (OOM), and any call `StackOverflowError` (Overflow). Graal IR gives a call an edge for its exceptions, which the translation keeps, but an allocation none, so the first stays implicit.

Operations on values are total. That is what Graal IR gives us: Native Image checks a divisor, an array index, and the receiver of an access explicitly, before the operation itself, with a branch

$$\textsc{Op}\ \frac{}{\langle \bar{x} \coloneqq op(\bar{y}), \rho, \sigma\rangle \Downarrow \langle \rho[\bar{x} \mapsto [\![op]\!](\rho(\bar{y}))], \sigma\rangle}$$

$$\textsc{Get}\ \frac{\rho(y) = o \neq \mathsf{null}}{\langle x, h' \coloneqq \mathsf{get}^P_f(h, y), \rho, \sigma\rangle \Downarrow \langle \rho[x \mapsto \sigma(o, f),\ h' \mapsto \bullet], \sigma\rangle}$$

$$\textsc{Set}\ \frac{\rho(y) = o \neq \mathsf{null}}{\langle h' \coloneqq \mathsf{set}^P_f(h, y, z), \rho, \sigma\rangle \Downarrow \langle \rho[h' \mapsto \bullet], \sigma[(o, f) \mapsto \rho(z)]\rangle} \qquad \textsc{Null}\ \frac{\rho(y) = \mathsf{null}}{\langle \iota, \rho, \sigma\rangle \Downarrow \lightning}$$

$$\textsc{New}\ \frac{o = \sigma(\mathsf{alloc})}{\langle x, h' \coloneqq \mathsf{new}_C(h), \rho, \sigma\rangle \Downarrow \langle \rho[x \mapsto o,\ h' \mapsto \bullet],\ \sigma[\mathsf{alloc} \mapsto o + 1][(o, f) \mapsto 0_f]_f\rangle}$$

$$\textsc{OOM}\ \frac{}{\langle x, h' \coloneqq \mathsf{new}_C(h), \rho, \sigma\rangle \Downarrow \mathsf{exc}(o_{\mathrm{oom}}, \sigma)}$$

$$\textsc{Guard}\ \frac{\rho(y) = ff}{\langle \bar{h}' \coloneqq \mathsf{guard}_b(\bar{h}, y), \rho, \sigma\rangle \Downarrow \langle \rho[\bar{h}' \mapsto \bullet], \sigma\rangle} \qquad \textsc{Leave}\ \frac{\rho(y) = tt \qquad \langle b, \rho(\bar{y}_b), \sigma\rangle \Downarrow r}{\langle \bar{h}' \coloneqq \mathsf{guard}_b(\bar{h}, y), \rho, \sigma\rangle \Downarrow r}$$

$$\textsc{Call}\ \frac{\langle m, \rho(\bar{y}), \sigma\rangle \Downarrow \mathsf{ret}(\bar{v}, \sigma')}{\langle \bar{x}, e, \bar{h}' \coloneqq m(\bar{y}, \bar{h}), \rho, \sigma\rangle \Downarrow \langle \rho[\bar{x} \mapsto \bar{v},\ e \mapsto \mathsf{null},\ \bar{h}' \mapsto \bullet], \sigma'\rangle}$$

$$\textsc{Catch}\ \frac{\langle m, \rho(\bar{y}), \sigma\rangle \Downarrow \mathsf{exc}(o, \sigma')}{\langle \bar{x}, e, \bar{h}' \coloneqq m(\bar{y}, \bar{h}), \rho, \sigma\rangle \Downarrow \langle \rho[\bar{x} \mapsto \bar{0},\ e \mapsto o,\ \bar{h}' \mapsto \bullet], \sigma'\rangle}$$

$$\textsc{Overflow}\ \frac{}{\langle \bar{x}, e, \bar{h}' \coloneqq m(\bar{y}, \bar{h}), \rho, \sigma\rangle \Downarrow \langle \rho[\bar{x} \mapsto \bar{0},\ e \mapsto o_{\mathrm{soe}},\ \bar{h}' \mapsto \bullet], \sigma\rangle}$$

$$\textsc{Stopped}\ \frac{\langle m, \rho(\bar{y}), \sigma\rangle \Downarrow \lightning}{\langle \bar{x}, e, \bar{h}' \coloneqq m(\bar{y}, \bar{h}), \rho, \sigma\rangle \Downarrow \lightning}$$

$$\textsc{Seq}\ \frac{\langle \iota, \rho, \sigma\rangle \Downarrow \langle \rho', \sigma'\rangle \qquad \langle \bar{\iota}; \kappa, \rho', \sigma'\rangle \Downarrow r}{\langle \iota\, \bar{\iota}; \kappa, \rho, \sigma\rangle \Downarrow r} \qquad \textsc{Stop}\ \frac{\langle \iota, \rho, \sigma\rangle \Downarrow r \qquad r \text{ is } \mathsf{exc}(o, \sigma') \text{ or } \lightning}{\langle \iota\, \bar{\iota}; \kappa, \rho, \sigma\rangle \Downarrow r}$$

$$\textsc{Jump}\ \frac{\langle b, \rho(\bar{y}), \sigma\rangle \Downarrow r}{\langle \mathsf{jump}\ b(\bar{y}), \rho, \sigma\rangle \Downarrow r} \qquad \textsc{If}\ \frac{\rho(y) = tt \qquad \langle b_1, \rho(\bar{y}), \sigma\rangle \Downarrow r}{\langle \mathsf{if}\ y\ \mathsf{then}\ b_1\ \mathsf{else}\ b_2\ (\bar{y}), \rho, \sigma\rangle \Downarrow r}$$

$$\textsc{Return}\ \frac{}{\langle \mathsf{return}(\bar{y}), \rho, \sigma\rangle \Downarrow \mathsf{ret}(\rho(\bar{y}), \sigma)} \qquad \textsc{Throw}\ \frac{}{\langle \mathsf{throw}(e, \bar{h}), \rho, \sigma\rangle \Downarrow \mathsf{exc}(\rho(e), \sigma)}$$

$$\textsc{Trap}\ \frac{}{\langle \mathsf{trap}(\bar{h}), \rho, \sigma\rangle \Downarrow \lightning} \qquad \textsc{Block}\ \frac{b(\bar{x} : \bar{\tau}) = \bar{\iota}; \kappa \qquad \langle \bar{\iota}; \kappa, [\bar{x} \mapsto \bar{v}], \sigma\rangle \Downarrow r}{\langle b, \bar{v}, \sigma\rangle \Downarrow r}$$

Figure 3.4: Evaluation. An outcome r is $\mathsf{ret}(\bar{v}, \sigma)$, $\mathsf{exc}(o, \sigma)$, an exception o with the heap the method leaves it in, or $\lightning$. An initialization, with a token or without, evaluates as Set, and exception as New, for the class of its exception; dup, drop and move only change the environment; Null applies to a load, a store and an initialization through null; if with $\rho(y) = ff$ goes to b_2; a guard passes b the values $\bar{y}_b$ the branch it abbreviates passed; a method is invoked as its entry block, with $\bullet$ for each token, $\langle m, \bar{v}, \sigma\rangle \Downarrow r$ if $\langle b_m, \bar{v}\ \bar{\bullet}, \sigma\rangle \Downarrow r$, and ret gives back only the values. 0_f is the default value of field f, New also sets the header fields, and o_{oom} and o_{soe} are an `OutOfMemoryError` and a `StackOverflowError` the virtual machine makes.

to a block that makes the exception and throws it. So an access never goes through null in our programs, and Null never applies.

Lemma 3.5 (Determinism)**.** *Which allocations and calls throw by OOM and Overflow determines the outcome: for every method m, values $\bar{v}$, heap σ and such a choice, there is at most one outcome r with $\langle m, \bar{v}, \sigma\rangle \Downarrow r$.*

Proof. By induction on the derivation: once the choice is made, the premises of every rule determine its conclusion, and an allocation takes the object $\sigma(\mathsf{alloc})$. □

Lemma 3.6 (Tokens carry nothing)**.** *Let two programs differ only in their tokens: in which token variables their instructions and terminators take and give, and in moves of tokens. Then every invocation has the same outcome in both.*

Proof. Every token variable holds •, and no rule looks at the value of a token. By induction on the derivation. □

The lemma has a consequence the rest of the chapter relies on: tokens add no behaviour to a program. A program annotated with the tokens of one partitioning or of another (Section 3.4) is the same program, with the same outcomes. What a partitioning changes is which rewrites its tokens license. In particular, the tokens order operations, but they do not hold the heap: an interpreter that gave each token a copy of its partition would be wrong for the programs of this thesis, which leave the tokens out of some initializations (Section 3.7).

Definition 3.7 (Runs)**.** The *run* of an invocation $\langle m, \bar{v}, \sigma\rangle$ is the sequence of *steps* $s_1, s_2, \ldots$ that its evaluation executes in the blocks of m itself, through jumps and branches: step s_k executes an instruction ι_k in environment ρ_k and heap σ_k, and leaves the heap σ_{k+1}. A call is one step. The run ends with the terminator that gives the outcome; the run of an invocation that does not terminate is infinite, or ends with a call that does not terminate. A step *takes* P if its instruction takes a token of P. A location *exists* at step k if its object exists in σ_k; objects are never removed, so it exists at every later step too.

The results below are about runs of the program: the invocation of its `main` method from the initial heap, and every invocation it makes. That is the only place where the facts of the points-to analysis hold.

3.3 Partitions

A partition is a set of locations, and a partitioning divides all of memory into disjoint partitions. Which partitions are useful depends on what is known about the objects an operation may touch, and that knowledge comes from a points-to analysis.

3.3.1 Abstract objects

A points-to analysis describes the objects a reference may point to by *abstract objects*. Native Image's analysis [20] describes an object by its class by default, and with allocation sites by the place in the program where it was allocated. We write $\alpha(o) \in AObj$ for the abstract object of o. It is determined when o is allocated, by the header fields cls and site, and never changes.

Assumption 3.8 (Sound points-to)**.** Each variable y that an access uses as its receiver has a set $pts(y) \subseteq AObj$, or $pts(y) = \top$ when the analysis cannot say. In every run of the program, whenever an access through y executes with $\rho(y) = o \neq \mathsf{null}$, then $pts(y) = \top$ or $\alpha(o) \in pts(y)$.

3.3.2 Partitionings

Definition 3.9 (Partitioning)**.** A *partitioning* is a set Π of nonempty, pairwise disjoint sets of locations, *partitions*, whose union is Loc, such that the partition of a field (o, f) depends only on f and $\alpha(o)$. We write $[\![\ell]\!]$ for the partition that contains ℓ, and rest for the one that contains alloc.

Since $\alpha(o)$ never changes, a location never changes its partition.

Definition 3.10 (Labelling)**.** A *labelling* gives each access s of a program, a load, a store or an initialization of field f, a partition $label(s) \in \Pi$. It is *sound* if in every run of the program every execution of s accesses a location of $label(s)$.

The four partitionings of the implementation differ in which locations must share a partition. Let F_p be the fields of the program's classes; every other field is the platform's, from the JDK or from Native Image. For $f \in F_p$, let $Known(f)$ be the abstract objects that the accesses of f through references the analysis can describe may touch: the union of $pts(y)$ over those accesses. An *abstract field* is a pair (f, a) with $a \in Known(f)$, or $(f, ?)$, which stands for every object no such access names. The abstract field of a location (o, f) is $(f, \alpha(o))$ if $\alpha(o) \in Known(f)$, and $(f, ?)$ otherwise. An access s of $f \in F_p$ through y *may touch* the abstract fields $E(s) = \{f\} \times pts(y)$, or every abstract field of f if $pts(y) = \top$.

Definition 3.11 (The four partitionings)**.** For an equivalence $\equiv$ on abstract fields, let $\Pi_\equiv$ be the partitioning in which two fields (o, f) and (o', f') with $f, f' \in F_p$ are in one partition if and only if their abstract fields are related by $\equiv$, and the platform's fields, the header fields and alloc form one more partition, rest. Let

- G_{acc} relate any two abstract fields that one access may touch,
- G_{obj} relate (f, a) and (g, a) for every abstract object a and fields f, g that accesses name it with: the fields of one object,

Partitioning	A partition of the program's fields is	Tells apart
none	all of memory, in one	nothing
field	one field of all objects	fields, as Graal's own locations do
object	every field of a set of objects	objects one access never mixes
object-field	one field of a set of objects	both

Table 3.1: The four partitionings. Under every one but none, the platform's fields and the allocator are one more partition, rest.

- G_{fld} relate any two abstract fields of one field f: one field of all objects,

and write G^* for the least equivalence that contains G. Then *object-field* is $\Pi_{\equiv}$ for $\equiv = G^*_{\text{acc}}$, *object* for $(G_{\text{acc}} \cup G_{\text{obj}})^*$, and *field* for $(G_{\text{acc}} \cup G_{\text{fld}})^* = G^*_{\text{fld}}$; *none* is $\{Loc\}$. The label of an access s of $f \in F_p$ is the partition of the abstract fields in $E(s)$, the label of an initialization is the partition of the abstract field of the location it initializes, and the label of an access of a platform field is rest.

Table 3.1 sums them up. The label is well defined because G_{acc} relates all abstract fields in $E(s)$, so they lie in one partition. The implementation computes object and object-field by union-find over the abstract fields the facts list [17]: it joins the abstract fields of each access, and for a field that some access touches through a reference the analysis can't describe, it joins $(f, ?)$ with every object the other accesses of that field may touch. That gives the same partitions as the definition for every location an access may touch.

Proposition 3.12 (Sound labelling)**.** *Under Assumption 3.8, each of the four is a partitioning, and its labelling is sound.*

Proof. The partitions are the classes of an equivalence, and rest, so they are disjoint and cover Loc, and the partition of (o, f) depends only on $(f, \alpha(o))$. Let an execution of an access s of field f through y access (o, f). If f is a platform field, $label(s) = \text{rest} \ni (o, f)$. Otherwise the abstract field of (o, f) is in $E(s)$: if $pts(y) = \top$, $E(s)$ holds every abstract field of f, and otherwise $\alpha(o) \in pts(y) \subseteq Known(f)$ by Assumption 3.8, so the abstract field is $(f, \alpha(o)) \in E(s)$. Hence the partition of (o, f) is that of $E(s)$, which is $label(s)$. An initialization is labelled by its location's partition. □

3.3.3 Finer and coarser

Definition 3.13 (Refinement)**.** A partitioning Π' *refines* Π, written $\Pi' \sqsubseteq \Pi$, if every partition of Π' is a subset of a partition of Π. For $P' \in \Pi'$ we write $\lceil P' \rceil$ for that partition of Π.

Proposition 3.14 (The order of the four)**.** *object-field* $\sqsubseteq$ *field* $\sqsubseteq$ *none, and object-field* $\sqsubseteq$ *object* $\sqsubseteq$ *none, while field and object are in general incomparable. Whenever* $\Pi' \sqsubseteq \Pi$ *are two of them, every access* s *has* $label_{\Pi'}(s) \subseteq label_{\Pi}(s)$*, that is,* $\lceil label_{\Pi'}(s) \rceil = label_{\Pi}(s)$*.*

Proof. If $G \subseteq G'$, then every class of G^* lies within a class of G'^*, and G_{acc} is contained in both of the larger relations. rest is the same set in all four, and everything lies within the one partition of none. The label of s under Π' is the partition that contains $E(s)$, which lies within the partition of Π that contains $E(s)$. For incomparability, see Example 3.15. □

Example 3.15 (Field and object are incomparable)**.** The method `deadStoreOtherField` of `Examples` stores to `value`, then to `other`, then to `value` again, all on one counter. With a partition per field, the store to `other` takes another token, so the first store's token goes straight to the third store, which overwrites it: the first store is dead (Section 3.6). With a partition per set of objects, all three take the counter's token, and the store to `other` stands in between. Conversely, in the running example `counter` and `other` are told apart only by the objects. The finest of the four, object-field, tells apart what either of them does.

3.4 Signatures

A method's signature says which partitions it, and everything it calls, may read and write. It is made of facts about each method, which come from the points-to analysis and from Graal IR (Chapter 5): the accesses it makes, with their receivers; the methods each of its calls may invoke; whether it allocates; and every other node of Graal IR that touches memory, with the locations Graal says the node touches.

Assumption 3.16 (Sound facts)**.**

1. For every call s, the set $\mathit{callees}(s)$ contains every method the call invokes in a run of the program.
2. Every other node that touches memory, other than an access, an allocation or a call, touches only locations in the partitions of fields and array kinds the facts list among its method's reads and writes, or makes its method opaque.

A method is *opaque* if it contains a node that may touch a location the facts cannot name (a location Graal does not name, a lock, an atomic update, a volatile access, a transition to native code), a call the analysis could not resolve, or a call of an opaque method.

Definition 3.17 (Signature)**.** Let Π be a partitioning with a labelling. The *signature* of a method m is the pair $(R(m), W(m))$ of sets of partitions that is the least solution of the constraints

$$
\begin{aligned}
R(m) &\supseteq \{\mathit{label}(s) \mid s \text{ a load in } m\},\\
W(m) &\supseteq \{\mathit{label}(s) \mid s \text{ a store in } m\} \cup \{\mathsf{rest} \mid m \text{ allocates}\},\\
R(m) &\supseteq R(m') \text{ and } W(m) \supseteq W(m') \quad \text{for every } m' \in \mathit{callees}(s) \text{ of a call } s \text{ in } m,
\end{aligned}
$$

together with the partitions of the locations the other nodes of m read and write. The *tokens* of m are $T_m = R(m) \cup W(m)$, or $T_m = \Pi$ if m is opaque.

The constraints are monotone, and their values form a finite lattice, so a least solution exists and iterating them from the empty sets reaches it [18]. The implementation computes it so (Chapter 5). Initializations are not among the stores: the facts leave them out, for the reason Section 3.7 gives.

Definition 3.18 (Token form)**.** The *token form* of a program under Π gives its instructions these tokens:

- every block of a method m takes the tokens of T_m;
- a load and a store take the token of their label;
- an initialization takes the token of its label if that is in T_m, and no token otherwise;
- an allocation takes rest;
- a call takes the tokens $T_{m'}$ of its callee m', the union of $T_{m'}$ over its callees if it has several, and all of Π if the analysis could not resolve it;
- a guard, and a node of the second kind in Assumption 3.16, takes the tokens T_m of its method.

Lemma 3.19 (The token form is well typed)**.** *The token form of every method is well typed.*

Proof. Every token an instruction of m takes is in T_m. For loads, stores and allocations, this is the first line of Definition 3.17. A callee's tokens are in T_m by the second line, and if the callee is opaque, so is m, and $T_m = \Pi$. An unresolved call makes m opaque too. The rest take T_m by definition. □

Example 3.20 (The running example)**.** The allocation-site analysis gives the two counters `main` passes to `forwardAcrossCallOnOtherObject` the abstract objects `Counter@main:199`, for `counter`, and `Counter@main:206`, for `other`. No access ever touches both, and `countInto` is only ever given the second. Under object-field, `Counter.value` therefore has two partitions, `value@main:199` and `value@main:206`, and

$$W(\texttt{countInto}) = \{\texttt{value@main:206}\},$$
$$R(\texttt{countInto}) = \{\texttt{value@main:206}, \texttt{Link.next}\}.$$

```
b2 = λ p0 p1 p2 h_c h_o h_n ⟶ r exc h_cout h_oout h_nout
(
    int cons_5 ⟶ v11
    heap set_0 h_c p0 v11 ⟶ h_c1                          ; counter.value = 5
    Aliasing_countInto run ⟶ k13
    f_rrhh_rhh call k13 p1 p2 h_o h_n ⟶ v17 h_o1 h_n1     ; countInto(other, list)
    ref nil? v17 ⟶ v17_none                               ; did it throw?
    ...                                                    ; b3 if not, b4 if it did
    f_rrhhh_irhhh call k v17 p0 h_c1 h_o1 h_n1 ⟶ r exc h_cout h_oout h_nout
)
b3 = λ v17 p0 h_c h_o h_n ⟶ r exc h_cout h_oout h_nout
(
    heap get_0 h_c p0 ⟶ v24 h_c1                          ; return counter.value
    ref null ⟶ exc
    int move v24 ⟶ r
    heap move h_c1 ⟶ h_cout
    heap move h_o ⟶ h_oout
    heap move h_n ⟶ h_nout
)
```

Figure 3.5: The running example under object-field, as the implementation prints it but without the copies of values or the branch on the exception, and with shorter names for the tokens: `h_c` for `Counter.value@main:199`, `h_o` for `Counter.value@main:206`, and `h_n` for `Link.next`. Field `o` is `Counter.value`.

Figure 3.5 shows the two blocks of the method that hold the store, the call and the load. The call takes the token of `other`'s counters and that of `Link.next`, and passes the token of `counter`'s straight from the store to the load. Under field the call takes `h_value` and `h_next`, and with one partition it takes `h`: in both the load's token comes from the call.

3.4.1 What a signature covers

Definition 3.21 (Footprint)**.** An invocation *reads* a location if one of the steps of its run is a load of it, an allocation (which reads alloc), or a call whose invocation reads it, and it *writes* a location if one of its steps is a store or an initialization of it, an allocation (which writes alloc), or a call whose invocation writes it. The *footprint* of an invocation $\langle m, \bar{v}, \sigma \rangle$ is made of the locations that exist in σ, when it starts, that it reads and writes.

Theorem 3.22 (Signatures)**.** *Assume Assumptions 3.8 and 3.16, and let* Π *be one of the four partitionings. For every invocation* $\langle m, \bar{v}, \sigma \rangle$ *in a run of the program, every location in its footprint lies in a partition of* T_m*, and if* m *is not opaque, every location of its footprint that it writes lies in a partition of* $W(m)$*.*

Proof. If m is opaque, $T_m = \Pi$ covers every location, so let m not be opaque. We proceed by induction on the derivation of the invocation's outcome. Let ℓ exist in σ, and let a step of the

run write ℓ; reads are similar, with $R(m)$ in place of $W(m)$ and $\mathsf{rest} \in W(m)$ for the reads of an allocation.

- A store: by Proposition 3.12, $\ell \in \mathit{label}(s)$, and $\mathit{label}(s) \in W(m)$ by Definition 3.17.
- An initialization belongs to the allocation of its object, which comes before it in the same block (Section 3.7). So its object was allocated during the invocation, and did not exist in σ.
- An allocation, or making an exception, writes $\mathsf{alloc} \in \mathsf{rest}$, and $\mathsf{rest} \in W(m)$.
- A call of m': $m' \in \mathit{callees}(s)$ by Assumption 3.16, and m' is not opaque, as m is not. ℓ exists when the call starts, as it existed in σ, and the call's invocation writes it. Its derivation is a part of the derivation of m's invocation, so by the induction hypothesis, ℓ lies in a partition of $W(m') \subseteq W(m)$.
- Any other node: by Assumption 3.16. □

Two things are outside the theorem. The first is the objects an invocation allocates itself: their fields are not in the footprint, which is what lets the facts leave initializations out (Section 3.7). The second is memory that is not in *Loc*. Graal also names locations of the virtual machine, such as the pointers of a thread's allocation buffer, a thread's local variables, and memory outside the Java heap. The facts list the ones a method writes, for the kill sets of Section 3.9, but neither the signatures nor the token form model them, and the facts do not list the ones a method reads. So the tokens order the operations on Java's memory: fields, array elements and the allocator. They do not order a write and a read of memory outside the heap made by two different callees. In the image of Are We Fast Yet, 25 methods write such memory. No rewrite of this thesis moves a call, so none of them depends on it, but a scheduler on the token form would need a token for the virtual machine's memory too.

3.5 What the tokens guarantee

The first guarantee is that an instruction that does not take a partition's token leaves the partition alone. Only fields of objects allocated in the meantime may change, and no operation before them can have seen those.

Lemma 3.23 (Frame). *Let s_j and s_k, $j < k$, be steps of a run of an invocation of m, and $P \in T_m$, such that no step strictly between them takes P, except moves and guards. Then every location of P that exists after s_j has the same value in σ_k as in σ_{j+1}.*

Proof. Let $\ell \in P$ exist in σ_{j+1}. We show that no step $s_i, j < i < k$, writes ℓ. Moves, operations on values, copies and loads write nothing, and so do guards: a guard that holds goes on in a block that leaves the method on every path, so the run never reaches s_k. A store or an initialization that takes the token of another partition Q writes a location of Q, by Proposition 3.12, and $Q \cap P = \emptyset$. An initialization that takes no token writes a location whose partition is not in T_m, by Definition 3.18, so not ℓ, as $P \in T_m$. An allocation, or the making of an exception, that does not take P writes $\mathsf{alloc} \notin P$ and fields of an object that did not exist before. A call that does not take P calls methods m' with $P \notin T_{m'}$, none of them opaque. ℓ exists when the call starts, so by Theorem 3.22 the call writes ℓ only if ℓ lies in a partition of $W(m') \subseteq T_{m'}$, which it does not. □

The second guarantee is that two instructions that share no token can be swapped. Its proof needs a property of the semantics that separation logic calls the frame property [21, 12]: an instruction's behaviour depends only on the locations it reads.

Lemma 3.24 (Locality). *Let $\langle \iota, \rho, \sigma \rangle \Downarrow r$, and let $\hat{\sigma}$ agree with σ on every location that exists in σ and that the step reads, and on* alloc *if it allocates. Then $\langle \iota, \rho, \hat{\sigma} \rangle \Downarrow \hat{r}$, with the same choice of errors, where $\hat{r}$ is r with the heap σ' it leaves, normally or with an exception, replaced by the heap $\hat{\sigma}'$ that agrees with σ' on every location the step writes or allocates, and with $\hat{\sigma}$ everywhere else; $\hat{r} = \lightning$ if $r = \lightning$.*

Proof. By induction on the derivation of r. Each rule reads only the locations it names, which the step reads; with the same values for them, every premise has the same outcome, and the rule writes the same locations with the same values. □

We also need that a reference never points to an object that does not exist: in every heap of a run, the fields of every existing object hold null, numbers or existing objects, and so do the environments. This holds initially and every rule preserves it.

Theorem 3.25 (Independence). *Let $\iota_1\ \iota_2$ be adjacent instructions of a block of m, such that ι_2 uses no result of ι_1, and the two take no common token. Then from every environment and heap a run reaches before them, $\iota_1\ \iota_2$ and $\iota_2\ \iota_1$ have the same normal outcomes: one of them gives $\langle \rho', \sigma' \rangle$ if and only if the other does.*

Proof. Let $\iota_1\ \iota_2$ go from $\langle \rho, \sigma \rangle$ through σ_1 to $\langle \rho', \sigma' \rangle$, and let T_1 and T_2 be the tokens the two take. First, consider an initialization that takes no token: its location lies in a partition no other instruction of m or of its callees touches, since a load, store or call that touched it would put its partition into T_m (Definition 3.17), and the initialization would then take the token. So it commutes with every instruction but the allocation of its object, which gives ι_2 a value it uses. Otherwise:

1. *The footprints are disjoint.* By Proposition 3.12 for accesses and Theorem 3.22 for calls, every location that exists before a step and that it reads or writes lies in a partition the step takes. A location that exists in σ, and that one of them writes and the other reads or writes, would lie in a partition of $T_1 \cap T_2 = \emptyset$.

2. *At most one of them allocates*, since both would take rest.

3. ι_2 *never reaches an object* ι_1 *allocated.* Its arguments do not come from ι_1, so they are null, numbers or objects that exist in σ. Every reference it reads later comes from a field of an object that exists in σ, which ι_1 did not write by (1), so the field holds what it held in σ, an object that existed then; or from a field of an object ι_2 allocated itself.

4. By (1)–(3), σ and σ_1 agree on everything ι_2 reads, and on alloc if ι_2 allocates. By Lemma 3.24, ι_2 from $\langle \rho, \sigma \rangle$ gives the same values and the same writes, and reaches a heap σ_2.

5. In the same way, σ and σ_2 agree on everything ι_1 reads from σ, so ι_1 from σ_2 gives the same values and writes as from σ.

Both orders bind the same values and make the same writes, to disjoint locations, and allocate the same objects, so they end in the same environment and heap. The converse is symmetric. □

Corollary 3.26 (Any order)**.** *Any two orders of the instructions of a block that respect its data flow, of values and of tokens alike, have the same normal outcomes.*

Proof. The data flow orders the instructions partially, and two linear extensions of a partial order differ by a sequence of swaps of adjacent incomparable elements. Each swap preserves the normal outcomes by Theorem 3.25. □

Remark 3.27 (Abnormal ends)*.* The theorem says nothing about exceptions and nontermination, and cannot. A guard takes every token of its method, but a call of a method that touches no memory takes none, so the two share no token. If the guard holds and the callee never returns, swapping them turns an exception into a program that hangs. No rewrite of this thesis moves an instruction, so none depends on it. A scheduler on the token form would need termination, like memory, to be an effect with a token.

3.6 What the tokens license

Two rewrites follow from the guarantees: replacing a load by a value already known, and removing a store nothing reads. For each, we first say what must hold in every run, then what the token form shows.

3.6.1 Forwarding a load

Definition 3.28 (Forwardable load)**.** A load ι, x, $h' \coloneqq \mathsf{get}_f^P(h, y)$, of a method m is *forwardable* from a variable v bound before it in its block if in every run of an invocation of m, every step s_k that executes ι has an earlier step s_j such that

1. s_j is a load, a store or an initialization of field f of the object $o = \rho_k(y)$, and $\rho_k(v)$ is the value it read or wrote, and
2. no step strictly between s_j and s_k takes P, except moves and guards.

Theorem 3.29 (Forwarding)**.** *Replacing a forwardable load ι by $x \coloneqq \mathsf{move}\ v$; $h' \coloneqq \mathsf{move}\ h$ preserves the outcomes of the program.*

Proof. Let p' be the program with ι replaced. We show by induction on derivations, in both directions, that every invocation has the same outcome in p and in p', for the same choice of errors. Invocations of other methods than m differ only in the invocations they make, so consider one of m, and its run in p. Up to a step s_k that executes ι, p' executes the same steps, with the same environments and heaps. At s_k, p reads $\sigma_k(o, f)$ with $o = \rho_k(y)$. $o \neq \mathsf{null}$, since s_j accessed it, so Null does not apply. s_j left $\rho_k(v)$ in (o, f), which exists after s_j and lies in P, since ι accesses it (Proposition 3.12). By Lemma 3.23, $\sigma_k(o, f) = \sigma_{j+1}(o, f) = \rho_k(v)$. So x gets the same value in p and p', and neither changes the heap; from there on the runs are the same again. □

The proof never mentions exceptions: forwarding happens on the path on which nothing was thrown.

Lemma 3.30 (Straight tokens)**.** *Let a load ι of f through y take a token of P that comes, through moves and guards only, from the token an access ι_0 of f through y in the same block gives. Then ι is forwardable from the value ι_0 reads or writes.*

Proof. By Lemma 3.3, in every execution of the block, the steps between ι_0 and ι that take P are the moves and guards on the token's way. y is bound once, so both access the same object. □

Between blocks, the rules of Chapter 4 first pass the value to the load's block as a parameter, and then apply the lemma within the block (Section 3.6.4).

Example 3.31 (The running example, forwarded)**.** Under object-field, the rules join the three blocks of Fig. 3.5 into one and turn the null check into a guard on each token, and the branch on the exception `countInto` gave back into a guard too. The load's token then comes from the store, past a guard, and both go through `counter`, so by Lemma 3.30 the load is forwardable from the stored 5. In the names of Fig. 3.5:

```
heap set_0 h_c_g p0 v11 ⟶ h_c1                    ; counter.value = 5
Aliasing_countInto run ⟶ k13
f_rrhh_rhh call k13 p1 p2 h_o_g h_n_g ⟶ v17 h_o1 h_n1   ; countInto(other, list)
ref nil? v17 ⟶ v17_none
bool not v17_none ⟶ cn
heap guard h_c1 cn ⟶ h_c1_g   ; throws              ; leave if countInto threw
heap guard h_o1 cn ⟶ h_o1_g   ; throws
heap guard h_n1 cn ⟶ h_n1_g   ; throws
int move v11 ⟶ v24                                ; forwarded Aliasing$Counter.value
heap move h_c1_g ⟶ h_c2
```

Under field and with one partition, the load's token comes from the call, and the lemma does not apply.

3.6.2 Removing a store

Definition 3.32 (Overwritten store)**.** A store ι, $h' \coloneqq \mathsf{set}^{P}_{f}(h, y, z)$, of a method m is *overwritten* if in every run of an invocation of m, every step s_j that executes ι has $o = \rho_j(y) \neq \mathsf{null}$ and is followed by one of:

1. a step s_k that stores to (o, f), such that no step strictly between s_j and s_k takes P, except moves and guards;
2. the end of the run with $\lightning$, or no end at all, such that no step after s_j takes P, except moves and guards.

In particular, the run does not leave the method between a store that is overwritten and the store that overwrites it, neither by returning nor with an exception. Any call may throw `StackOverflowError` and any allocation `OutOfMemoryError`, so a call or an allocation between the two stores, on any token, keeps the first: in the run in which it throws, the method leaves with the first value in (o, f).

Theorem 3.33 (Dead stores)**.** *Removing an overwritten store, that is, replacing it by $h' \coloneqq \mathsf{move}\ h$, preserves the outcomes of the program.*

Proof. Let p' be the program without ι, and consider a run of m in p that executes ι at step s_j, and the run in p' with the same choice of errors. Up to s_j, p' executes the same steps. From s_{j+1} on, until the overwrite in case 1, the two runs execute the same steps, with heaps that differ at (o, f) alone. For that, it is enough that no step between s_j and s_k reads or writes (o, f), since by Lemma 3.24 every such step then does the same in both. A step that reads or writes (o, f) takes P: a load or a store of it by Proposition 3.12; a call by Theorem 3.22, since (o, f) exists when the call starts. An initialization that takes no token belongs to the allocation of its object, and o was allocated before

s_j. In case 1, s_k then stores the same value to (o, f) in both runs, after which their heaps are equal, and the rest of the runs are the same. In case 2, both runs end with $\lightning$, or neither ends. Neither run leaves the method between s_j and s_k, so no outcome, normal or exceptional, carries a heap in which the two differ. The converse is symmetric. □

Example 3.34 (Dead, and not dead)**.** In `deadStorePastOtherObject` of `Aliasing`,

```
int seen = other.other;
counter.value = 1;          // dead
seen += other.value;
counter.value = 2;
```

under object-field the load of `other.value` takes the token of `other`'s counters, so the first store's token goes straight to the second store. Apart from arithmetic, only the load comes between the two stores, and its receiver was checked for null before the first store, by the load of `other.other`. So the method cannot leave between them, and the first store is dead. In `deadStoreReadInArm` of `Examples`, once the load in a branch arm takes the value of the first store, the second store overwrites it, and the call `length(list)` comes between them. Under field the call takes only the token of `Link.next`, but it may throw `StackOverflowError`, and its exception goes to a block that leaves the method with it: a handler up the stack could read the first value, so the store is not dead.

A simpler model, in which a path that throws ends the program and nothing reads the heap after it, would make that store dead, and six more of `Examples`. That is not Java, so the token form makes exceptional paths explicit, as Graal IR does (Chapter 5), and the conditions below follow tokens into them.

3.6.3 Finer partitions license more

The rules check the conditions of this section on the token form, by following tokens.

Definition 3.35 (Static conditions)**.** In the token form of a method, a load is *straight* if it satisfies the condition of Lemma 3.30. A store of f through y that takes P is *statically overwritten* if every *P-path* from it ends at a store of f through y, or at a trap, and no path goes through a guard or past an allocation, on any token. A P-path follows the control flow from the store: within a block to the next instruction that takes P and is not a move or a guard, or to the terminator; through a jump or either arm of a branch to the start of the next block, following y into the parameter it is passed to. It ends there, or where the method leaves, at a return or a throw. A path that comes back to a block it went through, with the token and y in the same parameters, is cut. A call between the store and the overwrite needs no rule of its own: its block branches on its exception, and a path into the block that throws it ends at a throw.

A run that executes a statically overwritten store follows one of the P-paths of the store. By Lemma 3.3, the steps in which it takes P are the instructions on that path. A run that goes around a cut loop forever takes P in no step after the store. A guard that holds, or an allocation that throws, is the only way out of a path that the paths themselves don't show. So a statically overwritten store whose receiver is never null is overwritten.

Theorem 3.36 (Monotonicity). *Let $\Pi' \sqsubseteq \Pi$ be two of the four partitionings. Every load that is straight under Π is straight under Π', and every store statically overwritten under Π is statically overwritten under Π'.*

Proof. First, every instruction that takes a token of $P' \in \Pi'$ in the token form under Π' takes the token of $\lceil P' \rceil$ under Π:

- a load or a store by Proposition 3.14; an initialization too, since if its label P' is in T'_m, then $\lceil P' \rceil$, its label under Π, is in T_m, as for a call below;
- an allocation, since rest is the same partition in both;
- a call, a guard or another node, which take the tokens of a method n, its callee or its own: if $P' \in T'_n$, then either P' is the label under Π' of an access in a method n reaches, and $\lceil P' \rceil$ is that access's label under Π, which is in T_n; or n is opaque, under Π as well, since opacity does not depend on the partitioning.

Now let a load ι of f through y be straight under Π, from ι_0, with label P' under Π' and $P = \lceil P' \rceil$ under Π. ι_0 accesses f through y too, so its label under Π' is also P'. Under Π, the instructions between them that take P are moves and guards. An instruction between them that takes P' under Π' takes P under Π, so it is a move or a guard, and ι is straight under Π'. For a store, the P'-paths and the P-paths follow the same control flow, and on each, the instructions that take P' are among those that take P. The first one that is not a move or a guard on a P-path is a store of f through y, which takes P' too, so it is the first on the P'-path as well; or the path reaches a trap or is cut, and so does the P'-path. Where a path may leave the method, at a guard, an allocation, a return or a throw, does not depend on the partitioning: guards take every token of their method under both. □

The theorem is about the conditions. The rules apply them in some order, interleaved with changes of shape that copy code (Section 3.6.4), and the copies count against bounds: peeling a loop copies at most 40 instructions of its header and body. A finer partitioning has more tokens, so a null check becomes more guards, and a loop a coarser partitioning peels may be too long to peel under a finer one. So the implementation is not monotone by construction, and we checked that it is on the programs of this thesis. Table 3.2 shows, for each program, what the rules leave

Program	Methods	none	field	object	object-field
`Aliasing`	14	18/15	18/15	17/12	17/12
`Examples`	41	52/37	51/29	52/29	51/29
`Signatures`	18	10/10	10/6	10/6	10/6
Are We Fast Yet	370	223/345	223/308	223/321	223/308
the native-image driver	436	266/714	266/695	266/704	266/695

Table 3.2: The field stores and loads the rules leave in the translated methods, under each partitioning, on the export with allocation sites. No method keeps a store or a load under a finer partitioning that a coarser one removes.

under the four partitionings, all from the export with allocation sites, so that the program is the same under each. In no method does a finer partitioning keep a store or a load that a coarser one removes, counted by where it is in the bytecode, or keep more of them in its loops, and every store dead under a coarser partitioning is dead under a finer one. Counted in the code, one method keeps a store more under field than with one heap: `forwardAcrossIterationsPeeled`, whose loop field lets the rules peel. Peeling copies the loop's store, and both copies stay, since the call in the loop may throw before the next iteration overwrites them. A check of the test suite does the same on the three programs it uses.

3.6.4 From licences to rules

The rules of Chapter 4 do more than the two lemmas: they forward into the arms of a branch, into and out of loops, and across merges. They do it by changing the shape of the program first, so that the value reaches the load's block as a parameter, and then applying Lemma 3.30 within the block.

Lemma 3.37 (Shapes)**.** *Each of the following changes preserves the outcome of the program, and for every partition P, the sequence of steps that take P other than moves and guards, with their values:*

1. *a branch to a block every path from which leaves the method becomes a guard that goes there, on each token, and a jump to the other block;*
2. *a block that only one jump reaches joins the block that jumps;*
3. *a parameter of a block for which every jump and branch passes the same value as for another parameter is replaced by that one;*
4. *a block takes a new parameter, which every jump and branch to it passes;*
5. *the first iteration of a loop is peeled: copies of its header and body run it, and jump into the loop;*

6. *a merge is split: the blocks that jump to it are divided in two, one of them jumps to a copy of the merge, and both copies go on in a new block with the code after a given point.*

Proof sketch. A guard abbreviates the branch it replaces, so (1) changes nothing but how the program is written. Changes (2)–(6) rename and copy blocks: every run of the new program executes the same instructions, in the same order and with the same values, as a run of the old one, and the other way round. □

A rule then forwards a load by passing, as the new parameter of (4), the value of the access each jump's token comes from. In every run the parameter then holds the value of the access before the load, and the load is forwardable from it. For a load in a loop, this is the value the entry knows, or the value the body left the iteration before, a loop-carried φ.

3.7 Fresh objects

Graal gives the objects that escape analysis keeps virtual their first values without stores. When such an object escapes, a single node, the commit of an allocation, allocates it and gives each of its fields its first value. Graal's model of memory has a location of its own for such writes, `LocationIdentity.init()`, which no read ever names: a field of a new object can only be read through a reference to it, and the reference only exists once the commit is done.

The translation writes a commit as the allocations followed by an initialization of each field, and the facts leave initializations out of the signatures. So an initialization takes a token only when its method takes the token of the field's partition for another reason, and otherwise takes none.

Proposition 3.38 (Fresh objects)**.** *The results of this chapter hold for the token form with initializations that take no token.*

Proof. Each proof treats them as a case of its own: Theorem 3.22, because the object did not exist when the invocation started; Lemma 3.23, because the partition is not one of the method's; Theorem 3.25, because nothing else in the method or its callees touches the partition; and Theorem 3.33, because the stored object was allocated before the store. □

The exception a failed check makes is a new object too. The facts leave its allocation out, as Graal's do, so the implementation gives exception no token, where the core language gives it rest. Nothing before can have seen the exception; only the order in which objects are allocated could tell, which no rewrite of this thesis changes.

This is also why Lemma 3.23 is about the locations that exist after the earlier step. A callee may allocate an object, initialize a field of it in a partition of the caller, and give it back; that field then

changes between two steps of the caller that take the partition, without the call taking it. But no earlier operation can have seen the field, so no rewrite based on one can be about it.

Theorem 3.22 constrains only locations that exist when an invocation starts. So an access that, in every run, touches only fields of objects allocated during the current invocation of its method could be left out of the signature altogether: no caller can tell it happened. The method itself still has to order such accesses among themselves, with a token of its own that it creates where it allocates the objects and consumes before it returns, as Haskell's state threads do [9]. An initialization needs no such token, because when it takes none, nothing else in its method touches its partition. Leaving fresh objects out of a signature is the *prestate* of Sălcianu and Rinard's purity analysis [14], and the effect masking of Lucassen and Gifford's effect systems [10]. Telling which accesses touch only fresh objects takes an analysis of its own: an allocation site is not enough, since it allocates in every invocation, while Sălcianu and Rinard's analysis tells apart the objects allocated during the current one. The implementation uses freshness only where Graal already knows it, for commits.

3.8 Concurrency

The semantics of Fig. 3.4 is sequential: a run is one thread. Java threads share the heap, and another thread may write a location between two steps of a run. The Java memory model [11] promises sequential consistency to programs free of data races [1]: if no sequentially consistent execution of a program has two accesses to one location from two threads, at least one of them a write, that synchronization does not order, then every execution behaves as an interleaving of the steps of its threads. For such programs, removing a load that an earlier access to the same location in the same thread makes redundant, and a store that a later store overwrites, is safe when no synchronization comes between them [15]. For programs with data races, the guarantees of the Java memory model are weaker, and which transformations it allows is subtle [16]. We make no claims about such programs.

In the facts, every synchronization (a lock, a volatile access, an atomic update, a fence) makes its method opaque, so a call that synchronizes takes every token, and nothing is forwarded or removed across it. The translation refuses methods that make a volatile access themselves (Chapter 5).

That is more than necessary. Synchronization makes the writes of other threads visible, and other threads can only write locations they can reach.

Proposition 3.39 (Thread-local partitions)**.** *Let the program be free of data races, and let P be a partition whose locations no two threads access in any execution. Then Lemma 3.23, and with it the results that rely on it, hold for P even if a step that only synchronizes does not take P.*

Proof sketch. Every execution is an interleaving of the steps of its threads. Steps of other threads access no location of P, and a synchronization touches its own lock or volatile field, not a location

of P. So between two steps of a run, only steps of the same run write the locations of P, and the proof of Lemma 3.23 applies to them. □

Which partitions are thread-local is what a thread-escape analysis tells [4, 13]. That the program starts no thread is not enough: Native Image runs threads of its own, which handle references the program registers, such as cleaners. In Are We Fast Yet, after the kill sets were made precise, a lock is the first effect 333 of the 455 calls that still kill everything reach, 294 of them through the lock `Throwable.fillInStackTrace` takes on the exception it is building, which no other thread can see yet. Run-time class initialization, which runs code under a lock, accounts for 108 more (Chapter 6). This condition is not implemented.

3.9 What a kill set guarantees

Signatures need not go through tokens. Graal's own model of memory orders a read after the last node before it that kills the read's location, and takes a call to kill every location. The round trip of Chapter 5 gives each call a *kill set* instead: the locations its callees may write, made from the same facts.

Graal's locations are the fields, one location per field or, under object-field, per partition of a field; the array element kinds; init; the locations of the virtual machine; and any, which overlaps all of them. Every node that writes memory says which locations it kills.

Assumption 3.40 (Graal's model)**.** A node writes only locations it kills. The writes that initialize the objects a node allocates are kills of init.

Graal's own phases rely on this for every node, so we rely on it too.

Definition 3.41 (Kill set)**.** The kill set $K(m)$ of a method is any if m is opaque. Otherwise it is the least set that contains the locations of m's own writes, namely the field, or its partition, of a store; the element kind of an array store; the locations another node kills; and init if m allocates, together with $K(m')$ for every callee m' of every call in m. A call of m kills $K(m) \cup V$, where V holds init and the locations of the virtual machine that may change at any safepoint: the pointers of the thread's allocation buffer, the salt of identity hash codes, the safepoint counter, and the state of memory sessions.

Theorem 3.42 (Kill sets)**.** *Assume Assumptions 3.8, 3.16 and 3.40, and that the image never executes a call its analysis found unreachable. Then every location written during an invocation of m is covered by a location of $K(m) \cup V$, where a field of an object allocated during the invocation and given its first value by a commit counts as covered by* init.

Result	Assumes	Relied on by
Chains (3.3)	single use, well typed	everything below
Signatures (3.22)	sound points-to and facts	the frame, independence, dead stores
Frame (3.23)	signatures	forwarding, dead stores
Independence (3.25)	frame, locality	that tokens order what must be ordered
Forwarding (3.29)	frame	the rules' forwarding
Dead stores (3.33)	frame, the method can't leave in between	the rules' dead stores, which Native Image removes
Monotonicity (3.36)	refinement	choosing a partitioning
Fresh objects (3.38)	Graal's commits	signatures without initializations
Thread-local partitions (3.39)	no data races, a thread-escape analysis	not implemented
Kill sets (3.42)	Graal's model, the closed world	the round trip

Table 3.3: The results of this chapter.

Proof sketch. By induction on the derivation, as for Theorem 3.22, with Graal's locations in place of partitions: a store writes its field's location, or its partition's, which is in $K(m)$; another node writes what it kills, by Assumption 3.40; a call writes what its callee does, which the analysis lists, unless the call is unreachable, and the image then never makes it. What changes at a safepoint is written by the virtual machine, not by the callee: any call may reach a safepoint, and while it waits there, another thread may collect garbage and reset the allocation buffer. Native Image declares the same locations V at its own safepoints. □

A kill set covers writes only. A read in the callee changes nothing that a read in the caller could see, so the caller may still take a value from before the call. What a read in the callee does need is that a store before the call is neither removed nor moved past it. Graal keeps stores and calls in their order, and the only rule with which we saw it remove a dead store requires the two stores to be adjacent (Chapter 5). So an effect that only reads makes its method take every token, but leaves its kill set alone. The kill set is to a call what $W(m)$ is to its tokens: a call takes the tokens of $R(m) \cup W(m)$, but kills only the locations of $W(m)$.

3.10 Summary

Table 3.3 lists the results. Stating them precisely, and holding them against the implementation, turned up four things the earlier chapters take for granted:

- *The frame holds only for locations that already exist.* Between two operations on a partition, a callee may initialize a new object's field in it without taking its token. Nothing can have

seen the field before, so no rewrite is affected, but the tokens do not carry the heap: an interpreter that gave each token its partition's contents would be wrong.

- *Tokens model Java's memory, not the virtual machine's.* The facts list the virtual machine's locations a method writes only for the kill sets, and not the ones it reads. No rewrite moves a call, so none depends on it.
- *Independence holds for normal outcomes only.* Tokens do not order a guard that may leave the method before a call that may not return. No rewrite moves an instruction, so none depends on it.
- *Monotonicity holds of the conditions, not of the rules by construction*, since peeling is bounded by a number of instructions that grows with the number of tokens. It holds on every program of this thesis, and a check keeps it so.

4 Analysis and Optimization of the Explict Effect Form

5 From Graal IR and back

I went to hell and back just to find piece

6 Evaluation

7 Conclusion

Bibliography

[1] Sarita V. Adve and Mark D. Hill. "Weak Ordering – A New Definition". In: *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*. 1990.

[2] Andrew W. Appel. "SSA is Functional Programming". In: *ACM SIGPLAN Notices* 33.4 (1998).

[3] Erik Barendsen and Sjaak Smetsers. "Uniqueness Typing for Functional Languages with Graph Rewriting Semantics". In: *Mathematical Structures in Computer Science* 6.6 (1996).

[4] Jong-Deok Choi, Manish Gupta, Mauricio Serrano, Vugranam C. Sreedhar, and Sam Midkiff. "Escape Analysis for Java". In: *Proceedings of the 14th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. 1999.

[5] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. "Efficiently Computing Static Single Assignment Form and the Control Dependence Graph". In: *ACM Transactions on Programming Languages and Systems* 13.4 (1991).

[6] Gilles Duboscq, Thomas Würthinger, Lukas Stadler, Christian Wimmer, Doug Simon, and Hanspeter Mössenböck. "An Intermediate Representation for Speculative Optimizations in a Dynamic Compiler". In: *Proceedings of the 7th ACM Workshop on Virtual Machines and Intermediate Languages (VMIL)*. 2013.

[7] Jean-Yves Girard. "Linear Logic". In: *Theoretical Computer Science* 50.1 (1987).

[8] Richard A. Kelsey. "A Correspondence between Continuation Passing Style and Static Single Assignment Form". In: *Papers from the 1995 ACM SIGPLAN Workshop on Intermediate Representations (IR '95)*. 1995.

[9] John Launchbury and Simon L. Peyton Jones. "Lazy Functional State Threads". In: *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation (PLDI)*. 1994.

[10] John M. Lucassen and David K. Gifford. "Polymorphic Effect Systems". In: *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 1988.

[11] Jeremy Manson, William Pugh, and Sarita V. Adve. "The Java Memory Model". In: *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 2005.

[12] John C. Reynolds. "Separation Logic: A Logic for Shared Mutable Data Structures". In: *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS)*. 2002.

[13] Alexandru Sălcianu and Martin Rinard. "Pointer and Escape Analysis for Multithreaded Programs". In: *Proceedings of the Eighth ACM SIGPLAN Symposium on Principles and Practices of Parallel Programming (PPoPP)*. 2001.

[14] Alexandru Sălcianu and Martin Rinard. "Purity and Side Effect Analysis for Java Programs". In: *Verification, Model Checking, and Abstract Interpretation (VMCAI)*. Vol. 3385. Lecture Notes in Computer Science. 2005.

[15] Jaroslav Ševčík. "Safe Optimisations for Shared-Memory Concurrent Programs". In: *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2011.

[16] Jaroslav Ševčík and David Aspinall. "On Validity of Program Transformations in the Java Memory Model". In: *ECOOP 2008 – Object-Oriented Programming*. Vol. 5142. Lecture Notes in Computer Science. 2008.

[17] Bjarne Steensgaard. "Points-to Analysis in Almost Linear Time". In: *Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 1996.

[18] Alfred Tarski. "A Lattice-Theoretical Fixpoint Theorem and its Applications". In: *Pacific Journal of Mathematics* 5.2 (1955).

[19] Philip Wadler. "Linear Types Can Change the World!" In: *Programming Concepts and Methods*. North-Holland, 1990.

[20] Christian Wimmer, Codrut Stancu, Peter Hofer, Vojin Jovanovic, Paul Wögerer, Peter B. Kessler, Oleg Pliss, and Thomas Würthinger. "Initialize Once, Start Fast: Application Initialization at Build Time". In: *Proceedings of the ACM on Programming Languages* 3.OOPSLA (2019).

[21] Hongseok Yang and Peter O'Hearn. "A Semantic Basis for Local Reasoning". In: *Foundations of Software Science and Computation Structures (FoSSaCS)*. Vol. 2303. Lecture Notes in Computer Science. 2002.